

# Semi-Secret Wars

## Game Design Document (GDD)

**Version:** 0.1

**Status:** Living Document

---

### 1. Game Overview

#### Title

Semi-Secret Wars

---

#### Genre

Incremental Auto Battler

---

#### Engine

Godot (Latest Stable Version)

---

#### Target Platform

PC

---

#### Project Status

Prototype / Vertical Slice

---

### 2. High Concept

Semi-Secret Wars is a minimalist Incremental Auto Battler presented as the pages of a child's comic notebook.

The player assembles a roster of heroes, assigns pre-battle priorities, and watches battles play out automatically. Every battle contributes to permanent progression through experience and hero upgrades, encouraging continuous improvement over many runs.

---

### 3. Design Pillars

- Incremental Progression
  - Strategic Preparation
  - Automated Combat
  - Permanent Hero Growth
  - Simple, Charming Presentation
  - Easy to Learn
  - Long-Term Progression
- 

### 4. Target Experience

The player should feel that:

- Every battle contributes to progress.
  - Every upgrade matters.
  - Every run is slightly more successful than the previous one.
  - Strategy happens before combat.
  - Watching battles is satisfying and easy to understand.
- 

### 5. Gameplay Loop

1. Select a hero roster.
  2. Assign each hero a battlefield priority.
  3. Start the battle.
  4. Watch heroes fight automatically.
  5. Receive rewards.
  6. Upgrade heroes.
  7. Repeat.
-

## 6. Visual Direction

### Art Style

- Minimalist
- Hand-drawn
- Notebook aesthetic
- Child's comic book
- Simple colors
- Thick outlines
- Imperfect shapes

The game takes place inside the pages of a child's notebook.

---

### UI Style

The interface should match the notebook presentation.

Menus should remain simple and readable.

Initial development uses placeholder UI.

---

### Animation Style

Characters should intentionally animate in a crude but charming way.

Visual inspiration:

- South Park
- Paper puppets
- Children's doodles

Animations should primarily use:

- Body bobbing
- Small rotations
- Simple scaling
- Limited keyframes

The goal is personality rather than realism.

---

## 7. Heroes

### Initial Heroes

- Thundaar
- Artemis

Maximum party size:

#### 4 Heroes

Each hero has:

- Experience
- Levels
- Skill Tree
- Individual statistics

Detailed hero statistics:

**TBD**

Hero abilities:

**TBD**

---

## 8. Villains

Initial villains:

- Dark Mage
- Berserker

Each villain owns a unique battlefield.

Villains remain positioned away from the player's starting location.

Detailed villain statistics:

**TBD**

Villain abilities:

**TBD**

---

## 9. Minions

Minions exist to:

- Protect villains.
- Delay hero progression.
- Block objectives.
- Attack heroes.

Enemy types:

**TBD**

---

## 10. Battlefields

Each villain has one battlefield.

Battlefields include:

- Hero spawn location
- Objective locations
- Minion spawn areas
- Villain location

Battlefield layouts:

**TBD**

---

## 11. Objectives

Before battle, the player assigns priorities to each hero.

Possible priorities include:

- Attack Minions
- Capture Objectives
- Attack Villain
- Support Allies

Additional objective types:

**TBD**

---

## 12. Combat

Combat is fully automated.

Heroes should:

- Remove nearby threats.
- Continue toward assigned objectives.
- Progress toward the villain.

Minions attempt to stop heroes.

Villains defend the final area.

Combat formulas:

**TBD**

Damage calculations:

**TBD**

Status effects:

**TBD**

---

## 13. Progression

Heroes permanently gain:

- Experience
- Levels
- Skill Tree upgrades
- Stat improvements

Progress should be gradual and satisfying.

Every battle should contribute toward progression.

Experience formulas:

**TBD**

Level progression:

**TBD**

---

## 14. Skill Trees

Each hero owns an independent skill tree.

Skill trees should encourage long-term progression.

Node layout:

**TBD**

Upgrade costs:

**TBD**

Unlock conditions:

**TBD**

---

## 15. Game Economy

Rewards are earned after every battle.

Possible rewards include:

- Experience

Additional reward types:

**TBD**

Economy values:

**TBD**

---

## 16. Balance Philosophy

The game should reward persistence rather than luck.

Progression should feel:

- Predictable
- Fair
- Continuous
- Incremental

Players should notice gradual increases in power without trivializing earlier content.

Balance values remain configurable throughout development.

---

## 17. Placeholder Assets

During prototype development:

- Placeholder UI
- Placeholder characters
- Placeholder effects
- Placeholder environments

All placeholders should be easily replaceable.

Gameplay should never depend on final artwork.

---

## 18. Audio

Music:

**TBD**

Sound Effects:

**TBD**

Voice Acting:

**TBD**

---

## 19. Narrative

Story:

**TBD**

Setting:

A child's notebook.

Additional narrative information:

**TBD**

---

## 20. Localization

The project should support localization from the beginning.

Supported languages:

**TBD**

All player-facing text should eventually use localization keys.

---

## 21. Accessibility

Accessibility features:

**TBD**

---

## 22. Technical Constraints

Prototype development should prioritize:

- Simple systems
- Easy iteration
- Replaceable assets
- Modular content

Technical implementation details are documented separately in the AI Development Guide.

---

## 23. Future Expansion

Future features are intentionally left undefined until the prototype validates the core gameplay.

Future content:

**TBD**

---

## 24. Open Design Questions

This section tracks design decisions that remain unresolved.

Current questions:

- Combat formulas (**TBD**)
  - Hero base statistics (**TBD**)
  - Skill tree layouts (**TBD**)
  - Reward formulas (**TBD**)
  - Experience curve (**TBD**)
  - Number of minion types (**TBD**)
  - Battlefield layouts (**TBD**)
  - Localization languages (**TBD**)
- 

## 25. Design Principles

When making future design decisions, prioritize:

- Gameplay over presentation.
- Clarity over complexity.
- Incremental progression over sudden power spikes.
- Strategy before automation.
- Long-term scalability.
- Data-driven balancing.

The prototype should prove that the game is fun before expanding content or increasing production quality.

---

## Revision History

| Version | Date            | Notes                   |
|---------|-----------------|-------------------------|
| 0.1     | Initial version | Initial design document |